

Instructions

- Use **appropriate data structures** (List, Tuple, Set, Dictionary) as required.
 - Implement **selection statements** for decision-making.
 - Use **loops** effectively.
 - Apply **file handling** wherever specified.
 - Use **functions** to improve modularity and readability.
 - Handle exceptions appropriately during file operations.
-

Problem 1: Cyber Security Login Audit System

Problem Statement

A file named login_logs.txt contains user login attempts in the following format:

```
username,status
anuj,Success
rahul,Failed
anuj,Failed
priya,Failed
rahul,Failed
neha,Success
anuj,Failed
karan,Failed
rahul,Success
priya,Failed
```

Tasks

1. Count successful and failed login attempts.
2. Identify users with more than 2 failed attempts.
3. Create a dictionary storing the number of failures per user.
4. Create a set of users who logged in successfully.
5. Display users whose accounts should be reviewed.

Sample Output

Successful Login Attempts: 3

Failed Login Attempts: 7

Failure Count per User:

```
anuj : 2
rahul : 2
priya : 2
karan : 1
```

Users with Successful Logins:

```
{'anuj', 'neha', 'rahul'}
```

Accounts Requiring Review:

None

Problem 2: Airport Baggage Screening System

Problem Statement

Passenger baggage weights (in kg) are stored as tuples:

```
baggage = (  
    ("P101", 18),  
    ("P102", 32),  
    ("P103", 24),  
    ("P104", 36),  
    ("P105", 28),  
    ("P106", 20),  
    ("P107", 41),  
    ("P108", 26),  
    ("P109", 19),  
    ("P110", 34)  
)
```

Tasks

1. Display passengers carrying baggage above 30 kg.
2. Count passengers within and exceeding limits.
3. Calculate excess baggage charges (₹500 per kg above 30 kg).
4. Create a list of passengers requiring manual inspection.
5. Find the passenger carrying the heaviest baggage.

Sample Output

Passengers Exceeding 30 kg Limit:

P102
P104
P107
P110

Passengers Within Limit: 6

Passengers Exceeding Limit: 4

Excess Baggage Charges:

P102 : ₹1000
P104 : ₹3000
P107 : ₹5500
P110 : ₹2000

Passenger with Heaviest Baggage:

P107 (41 kg)

Passengers Requiring Manual Inspection:

['P102', 'P104', 'P107', 'P110']

Problem 3: E-Commerce Coupon Fraud Detection

Problem Statement

A file named coupons.txt contains coupon usage records.

```
SAVE50
WELCOME20
SAVE50
FESTIVE10
SAVE50
WELCOME20
NEWUSER
FESTIVE10
SAVE50
NEWUSER
```

Tasks

1. Count the usage frequency of each coupon.
2. Identify coupons used more than 3 times.
3. Create a set of unique coupons.
4. Display the most frequently used coupon.
5. Save suspicious coupon records into fraud_report.txt.

Sample Output

Coupon Usage Frequency:

```
SAVE50 : 4
WELCOME20 : 2
FESTIVE10 : 2
NEWUSER : 2
```

Suspicious Coupons:

```
SAVE50
```

Unique Coupons:

```
{'SAVE50', 'WELCOME20', 'FESTIVE10', 'NEWUSER'}
```

Most Frequently Used Coupon:

```
SAVE50
```

Fraud Report Generated Successfully.

Problem 4: Disaster Relief Resource Allocation

Problem Statement

Relief materials available at different warehouses are stored as dictionaries.

```
resources = {
    "Warehouse1": ["Food", "Medicine", "Blankets"],
```

```
"Warehouse2": ["Water", "Food", "Tents"],  
"Warehouse3": ["Medicine", "Tents", "Clothes"],  
"Warehouse4": ["Food", "Water", "Medicine"]  
}
```

Tasks

1. Display all unique relief items.
2. Find warehouses containing medicines.
3. Count how many warehouses stock each resource.
4. Identify the most widely available resource.
5. Display resources available in all warehouses.

Sample Output

Unique Resources:

{'Food', 'Medicine', 'Blankets', 'Water', 'Tents', 'Clothes'}

Warehouses with Medicines:

Warehouse1

Warehouse3

Warehouse4

Resource Availability:

Food : 3

Medicine : 3

Blankets : 1

Water : 2

Tents : 2

Clothes : 1

Most Widely Available Resources:

Food

Medicine

Resources Available in All Warehouses:

None

Problem 5: Digital Library Plagiarism Detector

Problem Statement

Two research abstracts are provided as strings.

abstract1 = "Artificial intelligence is transforming education and healthcare."

abstract2 = "Healthcare and education are rapidly transforming through artificial intelligence."

Tasks

1. Convert both abstracts into sets of words.
2. Identify common words.
3. Identify unique words in each abstract.

4. Calculate the percentage similarity.
5. Display whether plagiarism review is required (similarity > 50%).

Sample Output

Common Words:

{'artificial', 'intelligence', 'education', 'healthcare'}

Unique Words in Abstract 1:

{'is', 'transforming', 'and'}

Unique Words in Abstract 2:

{'are', 'rapidly', 'through', 'transforming'}

Similarity Percentage:

50.0%

Plagiarism Review Required:

No

Problem 6: Space Mission Telemetry Analyzer

Problem Statement

Sensor readings are stored in telemetry.txt.

101
98
105
110
112
95
90
88
120
102

Tasks

1. Read all sensor readings.
2. Display abnormal readings (< 90 or > 110).
3. Calculate average sensor value.
4. Count normal and abnormal readings.
5. Store abnormal readings in alerts.txt.

Sample Output

Abnormal Sensor Readings:

88
120

Average Sensor Value:

102.1

Normal Readings: 8

Abnormal Readings: 2

Alert File Generated Successfully.

Problem 7: Smart Agriculture Crop Monitoring System

Problem Statement

Crop moisture levels (%) are stored as follows:

```
moisture = {  
  "Field1": 55,  
  "Field2": 30,  
  "Field3": 72,  
  "Field4": 28,  
  "Field5": 64,  
  "Field6": 35,  
  "Field7": 80,  
  "Field8": 42,  
  "Field9": 25,  
  "Field10": 68  
}
```

Tasks

1. Identify fields requiring irrigation (< 40%).
2. Classify fields into Low, Moderate, and High moisture categories.
3. Count fields in each category.
4. Find fields with the highest and lowest moisture levels.
5. Generate an irrigation priority list.

Sample Output

Fields Requiring Irrigation:

Field2
Field4
Field6
Field9

Low Moisture Fields:

['Field2', 'Field4', 'Field6', 'Field9']

Moderate Moisture Fields:

['Field1', 'Field5', 'Field8']

High Moisture Fields:

['Field3', 'Field7', 'Field10']

Field with Highest Moisture:

Field7 (80%)

Field with Lowest Moisture:

Field9 (25%)

Irrigation Priority List:

['Field9', 'Field4', 'Field2', 'Field6']

Problem 8: Hospital Emergency Triage System

Problem Statement

Patients arriving at the emergency ward are categorized as:

```
patients = [  
    ("P101", "Critical"),  
    ("P102", "Stable"),  
    ("P103", "Critical"),  
    ("P104", "Moderate"),  
    ("P105", "Stable"),  
    ("P106", "Critical"),  
    ("P107", "Moderate"),  
    ("P108", "Stable"),  
    ("P109", "Critical"),  
    ("P110", "Moderate")  
]
```

Tasks

1. Count patients in each category.
2. Display IDs of critical patients.
3. Create separate lists for Critical, Moderate, and Stable patients.
4. Determine which category requires maximum attention.
5. Save critical patient IDs to critical_patients.txt.

Sample Output

Patient Count by Category:

Critical : 4

Moderate : 3

Stable : 3

Critical Patients:

P101

P103

P106

P109

Critical Patients List:

['P101', 'P103', 'P106', 'P109']

Moderate Patients List:

['P104', 'P107', 'P110']

Stable Patients List:

['P102', 'P105', 'P108']

Category Requiring Maximum Attention:

Critical

Critical Patient Report Generated Successfully.

Problem 9: Social Media Trend Analyzer

Problem Statement

Trending hashtags collected during an event are stored in a file named `hashtags.txt`.

#AI
#Python
#AI
#MachineLearning
#DataScience
#Python
#AI
#Coding
#DataScience
#Python

Tasks

1. Count occurrences of each hashtag.
2. Display the top trending hashtag.
3. Create a set of unique hashtags.
4. Identify hashtags used more than twice.
5. Generate a trend report file.

Sample Output

Hashtag Frequency:

#AI : 3
#Python : 3
#MachineLearning : 1
#DataScience : 2
#Coding : 1

Top Trending Hashtags:

#AI
#Python

Unique Hashtags:

{'#AI', '#Python', '#MachineLearning', '#DataScience', '#Coding'}

Hashtags Used More Than Twice:

#AI
#Python

Trend Report Generated Successfully.

Problem 10: Smart Traffic Signal Optimization System

Problem Statement

Vehicle counts recorded at a junction every 15 minutes are stored as follows:

traffic = [120, 95, 140, 180, 75, 60, 200, 160, 110, 85]

Tasks

1. Classify traffic conditions:
 - Low (< 80 vehicles)
 - Moderate (80–150 vehicles)
 - High (> 150 vehicles)
2. Count occurrences of each traffic condition.
3. Find the peak traffic interval.
4. Create separate lists for each traffic category.
5. Recommend whether manual traffic control is required (more than 3 High traffic intervals).

Sample Output

Traffic Conditions:

120 → Moderate
95 → Moderate
140 → Moderate
180 → High
75 → Low
60 → Low
200 → High
160 → High
110 → Moderate
85 → Moderate

Low Traffic Intervals: 2

Moderate Traffic Intervals: 5

High Traffic Intervals: 3

Peak Traffic Count:

200 vehicles

Low Traffic List:

[75, 60]

Moderate Traffic List:

[120, 95, 140, 110, 85]

High Traffic List:
[180, 200, 160]

Manual Traffic Control Required:
No
